AMAZON SALES



Business Analytics

Turning Transaction Data into Business Insights Using SQL

Rahul Chowdhury · 2026



Executive Summary

This portfolio project analyses the amazon_sales transactional dataset through 25 business questions. The analysis progresses from foundational sales KPIs to product and category performance, customer value, time-series trends, geographic comparisons, repeat purchasing, revenue concentration, and new-versus-returning customer revenue.

Evidence Note: The SQL queries were executed against the dataset, and verified execution outputs are included selectively throughout this report. Where queries return large result sets, only relevant or representative results are displayed for readability. Numerical findings and business insights are based on the verified outputs presented in the report.

Project Overview

The project is built on a single PostgreSQL table named amazon_sales. The table combines order, customer, product, financial, fulfilment and geographic dimensions in one transactional structure.

Dataset Schema

- **order_id** — order identifier
- **order_date** — transaction date and time-series analysis
- **customer_id / customer_name** — customer-level analysis
- **product_id / product_name** — product analysis
- **category / brand** — product segmentation
- **quantity** — sales volume
- **unit_price / discount / tax / shipping_cost** — transaction economics
- **total_amount** — revenue and customer-spending measure used in the queries
- **payment_method / order_status** — payment and fulfilment attributes
- **city / state / country** — geographic analysis
- **seller_id** — seller dimension available in the source table

Tools & Technology

PostgreSQL · SQL

Reporting Note

The SQL queries were executed against the complete dataset. For readability, only selected results are displayed in this report. Where a query returns a large number of records, the report may present only the top-ranking results, a limited sample, or selected rows from the full output. Any LIMIT clauses shown in the SQL define the specific subset being analysed, while other displayed excerpts are included solely to keep the report concise.

Analytical Framework

- **Sales Performance:** Q1, Q2, Q3
- **Product Performance:** Q4, Q9, Q11, Q16
- **Category Performance:** Q5, Q6, Q7
- **Payment Method Analysis:** Q8
- **Customer Analysis:** Q10, Q13, Q15, Q18, Q19, Q22, Q23
- **Geographic Analysis:** Q14
- **Seller Analysis:** Q17
- **Time-Based Sales Analysis:** Q12, Q20, Q21
- **Revenue Concentration & Retention:** Q24, Q25

1. Overall Sales Performance

Q1. How many orders were placed?

Analytical Approach: COUNT(DISTINCT order_id) measures the number of unique orders.

```
select count(distinct order_id) as order_count
from amazon_sales;
```

Result: A total of 100,000 unique orders were placed during the period covered by the dataset.

Business Insight: The dataset contains a substantial volume of 100,000 unique orders, providing a broad transaction base for analyzing sales and customer purchasing patterns.

Q2. What is the Average Order Value?

Analytical Approach: Calculates the average total_amount recorded in the table.

```
select round(avg(total_amount), 2) as avg_order_value
from amazon_sales;
```

Result: The average value of an order is 918.26, indicating that customers spend approximately 918.26 per order on average.

Business Insight: The 918.26 AOV provides a baseline for evaluating customer spending and comparing order values across categories, payment methods, and other customer segments.

Q3. How many units were sold in total?

Analytical Approach: The analysis calculates the total number of units sold by summing the quantity values across all orders in the dataset.

```
select sum(quantity) as total_units_sold
from amazon_sales;
```

Result: A total of 300,140 units were sold across the dataset.

Business Insight: The 300,140 units sold provide a baseline for measuring overall sales volume and product demand.

2. Product Performance

Q4. Top 10 products by Quantity Sold.

Analytical Approach: Aggregates quantity by product and returns the ten highest-volume products.

```
select product_id, product_name, sum(quantity) as quantity_sold
from amazon_sales
group by product_id, product_name
order by quantity_sold desc
limit 10;
```

Result: The top-selling product by quantity was the LED Desk Lamp, with 6,344 units sold. The top 10 products ranged from 6,139 to 6,344 units sold.

	product_id character varying (20)	product_name character varying (150)	quantity_sold bigint
1	P00019	LED Desk Lamp	6344
2	P00022	Water Bottle	6275
3	P00047	Memory Card 128GB	6240
4	P00037	Router	6202
5	P00032	Board Game	6200
6	P00040	Microphone	6196
7	P00006	Gaming Mouse	6170
8	P00017	Electric Kettle	6165
9	P00007	Mechanical Keyboard	6161
10	P00018	Vacuum Cleaner	6139

Business Insight: The top 10 products have fairly similar sales volumes, with only 205 units separating the highest- and lowest-ranked products. That suggests there isn't a single overwhelmingly dominant product among the top performers.

Q9. Find the highest-selling product (by Quantity Sold) within every category.

Analytical Approach: The analysis first aggregates the total quantity sold for each product within each category. A ROW_NUMBER() window function then ranks these products separately within their respective categories, ordering them by quantity sold in descending order. Finally, only the products assigned rank 1 are selected, giving the highest-selling product for every category.

```sql
with quantity_sales as(
    select category, product_id, product_name, sum(quantity) as quantity_sold
    from amazon_sales
    group by category, product_id, product_name
    ),
ranked_products as(
    select row_number() over(partition by category order by quantity_sold desc) as sales_rank,
    product_id, product_name, category, quantity_sold
    from quantity_sales
    )
select * from ranked_products
where sales_rank = 1;
```

Result: The highest-selling product was the Portable SSD 1TB in the Electronics category, with 1,135 units sold. Each category had a different top-performing product, with quantities ranging from 1,109 to 1,152 units.

	sales_rank bigint	product_id character varying (20)	product_name character varying (150)	category character varying (50)	quantity_sold bigint
1	1	P00023	Cookware Set	Books	1122
2	1	P00031	Kids Toy Car	Clothing	1109
3	1	P00009	Portable SSD 1TB	Electronics	1135
4	1	P00013	Action Camera	Home & Kitchen	1138
5	1	P00048	Wireless Charger	Sports & Outdoors	1152
6	1	P00019	LED Desk Lamp	Toys & Games	1140

Business Insight: The top-selling products vary across categories, indicating that no single product dominates sales across the entire dataset. Wireless Charger recorded the highest category-level quantity at 1,152 units in Sports & Outdoors, while Kids Toy Car had the lowest category-leading volume at 1,109 units.

Q11. Find the top 3 products (by Quantity Sold) within every category.

Analytical Approach: The analysis first aggregates the total quantity sold for each product within each category. A ROW_NUMBER() window function is then used to rank products separately within each category based on quantity sold in descending order. Finally, products ranked 1, 2, and 3 are selected to identify the top three best-selling products in every category.

```sql
with quantity_sales as (
    select product_id, product_name, sum(quantity) as quantity_sold, category
    from amazon_sales
    group by product_id, product_name, category
    ),
ranked_products as(
    select row_number() over(partition by category order by quantity_sold desc) as sales_rank,
    product_id, product_name, category, quantity_sold
    from quantity_sales
    )
select * from ranked_products
where sales_rank in (1,2,3);
```

Result: The analysis identified the top three products by quantity sold within each category. Wireless Charger in Sports & Outdoors recorded the highest quantity sold among all ranked products, with 1,152 units, followed by Jeans with 1,148 units in the same category.

	sales_rank bigint	product_id character varying (20)	product_name character varying (150)	category character varying (50)	quantity_sold bigint
1	1	P00023	Cookware Set	Books	1122
2	2	P00003	Smartphone Case	Books	1120
3	3	P00025	Running Shoes	Books	1112
4	1	P00031	Kids Toy Car	Clothing	1109
5	2	P00017	Electric Kettle	Clothing	1106
6	3	P00020	Office Chair	Clothing	1103
7	1	P00009	Portable SSD 1TB	Electronics	1135
8	2	P00024	Yoga Mat	Electronics	1133
9	3	P00036	Smart Light Bulb	Electronics	1108
10	1	P00013	Action Camera	Home & Kitchen	1138
11	2	P00009	Portable SSD 1TB	Home & Kitchen	1115
12	3	P00037	Router	Home & Kitchen	1106
13	1	P00048	Wireless Charger	Sports & Outdoors	1152
14	2	P00028	Jeans	Sports & Outdoors	1148
15	3	P00040	Microphone	Sports & Outdoors	1136
16	1	P00019	LED Desk Lamp	Toys & Games	1140
17	2	P00008	4K Monitor	Toys & Games	1111
18	3	P00037	Router	Toys & Games	1089

Business Insight: The top-performing products differ across categories, with Wireless Charger achieving the highest quantity sold overall at 1,152 units. The relatively narrow range among most top-three products indicates that sales volume is distributed fairly evenly among leading products rather than being dominated by a single product.

Q16. Find products purchased by the largest number of distinct customers.

Analytical Approach: The analysis groups the dataset by product and counts the distinct customers who purchased each product. This measures customer reach rather than the number of units sold or total orders. The products are then sorted in descending order by customer count, and LIMIT 1 returns the product purchased by the largest number of distinct customers.

```
select product_id, product_name, count(distinct(customer_id)) as customer_count
from amazon_sales
group by product_id, product_name
order by customer_count desc
limit 1;
```

Result: LED Desk Lamp (P00019) was purchased by 2,059 distinct customers, giving it the highest customer reach among the products in the dataset.

	product_id character varying (20)	product_name character varying (150)	customer_count bigint
1	P00019	LED Desk Lamp	2059

Business Insight: The LED Desk Lamp achieved the highest customer reach, being purchased by 2,059 distinct customers. This indicates that the product had the broadest customer adoption among the products in the dataset.

3. Category Performance

Q5. Which categories contribute the most to total revenue?

Analytical Approach: Calculates category revenue and its percentage of the overall category revenue.

```
with category_sales as (
    select
        category, sum(total_amount) as total_revenue
    from amazon_sales
group by category
    )
select category, total_revenue,
    round(total_revenue * 100 / sum(total_revenue) over(), 2) as pct_of_contrib
from category_sales;
```

Result: Electronics generated the highest revenue at 15,584,217.18, contributing 16.97% of total revenue. However, revenue was distributed fairly evenly across all six categories, with each category contributing between 16.51% and 16.97%.

	category character varying (50)	total_revenue numeric	pct_of_contrib numeric
1	Electronics	15584217.18	16.97
2	Sports & Outdoors	15345571.88	16.71
3	Toys & Games	15216684.99	16.57
4	Home & Kitchen	15163939.36	16.51
5	Clothing	15253397.50	16.61
6	Books	15261837.01	16.62

Business Insight: Electronics is the leading revenue category, but the difference between the highest and lowest contributors is only 0.46 percentage points. This indicates a highly balanced revenue distribution across categories, rather than strong dependence on a single category.

Q6. Which categories have the highest average order value (AOV)?

Analytical Approach: Compares average total_amount across categories and also reports distinct order count.

```sql
select category, count(distinct order_id) as order_count,
    round(avg(total_amount), 2) as aov_of_catg
    from amazon_sales
group by category
order by aov_of_catg desc;
```

Result: Clothing had the highest Average Order Value (AOV) at 927.88, followed by Electronics at 924.71. Books had the lowest AOV at 911.05.

	category character varying (50)	order_count bigint	aov_of_catg numeric
1	Clothing	16439	927.88
2	Electronics	16853	924.71
3	Toys & Games	16542	919.88
4	Sports & Outdoors	16804	913.21
5	Home & Kitchen	16610	912.94
6	Books	16752	911.05

Business Insight: AOV varies only modestly across categories, with a difference of 16.83 between the highest and lowest categories. Clothing therefore leads in average order value, but no category shows a dramatically higher order value than the others.

Q7. Which categories generate the most revenue relative to the quantity sold?

Analytical Approach: Sorts categories by total revenue in descending order, with quantity sold used as a secondary tie-breaker.

```sql
select category, sum(total_amount) as total_revenue, sum(quantity) as sold_qty
from amazon_sales
group by category
order by total_revenue desc, sold_qty asc;
```

Result: Electronics generated the highest revenue at 15,584,217.18, followed by Sports & Outdoors at 15,345,571.88. Clothing recorded the lowest unit sales at 49,523, while still generating 15,253,397.50 in revenue.

	category character varying (50)	total_revenue numeric	sold_qty bigint
1	Electronics	15584217.18	50696
2	Sports & Outdoors	15345571.88	50543
3	Books	15261837.01	49981
4	Clothing	15253397.50	49523
5	Toys & Games	15216684.99	49773
6	Home & Kitchen	15163939.36	49624

Business Insight: Revenue and unit sales were relatively balanced across categories. Electronics led in revenue despite having a similar sales volume to the other categories, while Clothing generated 15.25 million in revenue with the lowest unit volume of 49,523. This suggests that revenue differences across categories were not driven by large differences in units sold alone.

4. Payment Method Analysis

Q8. Which payment methods generate the highest revenue, and what is the average amount spent per order for each method?

Analytical Approach: Two CTEs were used to aggregate the data independently by payment method. The first calculates total revenue generated by each payment method, while the second calculates the total number of orders. These results are then joined using payment_method, and Average Order Value (AOV) is calculated by dividing total revenue by the corresponding order count. The results are sorted by total revenue in descending order.

```sql
with payment_revenue as (
    select
        payment_method,
        sum(total_amount) as revenue_generated
    from amazon_sales
    group by payment_method
),
payment_orders as (
    select
        payment_method,
        count(order_id) as order_count
    from amazon_sales
    group by payment_method
)
select
    r.payment_method,
    r.revenue_generated,
    o.order_count,
    round(r.revenue_generated / o.order_count, 2) as aov
from payment_revenue as r
left join payment_orders as o
    on r.payment_method = o.payment_method
order by r.revenue_generated desc;
```

Result: Credit Card generated the highest revenue at 32,212,158.69, with 35,038 orders and an AOV of 916.78. Debit Card recorded the highest AOV at 925.82, while Cash on Delivery had the lowest order volume at 4,928 orders.

	payment_method character varying (50)	revenue_generated numeric	order_count bigint	aov numeric
1	Credit Card	32122158.69	35038	916.78
2	Debit Card	18538678.53	20024	925.82
3	UPI	13896028.55	15066	922.34
4	Amazon Pay	13697498.42	15017	912.13
5	Net Banking	9055674.57	9927	912.23
6	Cash on Delivery	4515609.16	4928	916.32

Business Insight: Debit Card users generated the highest average spending per order, while Credit Card was the most frequently used payment method by order volume. This indicates that payment-method popularity does not necessarily correspond to higher average transaction value.

5. Customer Analysis

Q10. Identify customers who placed more than one order. Show their total number of orders and total spending.

Analytical Approach: Groups transactions by customer and filters to customers with more than one order.

```
select customer_id, customer_name, count(distinct order_id) as total_orders,
sum(total_amount) as total_spending from amazon_sales
group by customer_id, customer_name
having count(distinct order_id)>1;
```

Result: The analysis identified customers who placed more than one order and calculated their total spending. The displayed results show that these customers placed 2 orders each, with total spending ranging from 334.22 to 2,850.11 among the 10 displayed records.

	customer_id character varying (20)	customer_name character varying (100)	total_orders bigint	total_spending numeric
1	CUST000116	Rohit Gupta	2	2850.11
2	CUST000339	Priya Patel	2	1377.39
3	CUST000515	Mohit Sharma	2	977.24
4	CUST000526	Aditya Patel	2	1906.06
5	CUST000532	Sunita Kapoor	2	334.22
6	CUST000639	Priya Patel	2	2232.30
7	CUST000956	Ritika Mehta	2	2155.77
8	CUST000958	Sneha Verma	2	1271.61
9	CUST001009	Aman Kumar	2	302.38
10	CUST001159	Rohit Kapoor	2	1443.46

Business Insight: The results show that repeat customers contribute meaningful additional revenue, with spending varying considerably between customers despite the displayed customers having the same number of orders.

Q13. Find customers whose total spending exceeds the average customer spending.

Analytical Approach: The analysis first calculates each customer's total spending by summing their order amounts. It then calculates the average spending across all customers to establish a benchmark. Finally, each customer's total spending is compared against this average, and only customers whose spending exceeds the benchmark are returned.

```
with customer_spending as (
    select
        customer_id, customer_name,
        sum(total_amount) as total_spending
    from amazon_sales
    group by customer_id, customer_name
),
average_customer_spending as (
    select avg(total_spending) as avg_spending
    from customer_spending
)
select
    c.customer_id, c.customer_name, c.total_spending,
    round(a.avg_spending, 2) as avg_spending
from customer_spending c
cross join average_customer_spending a
where c.total_spending > a.avg_spending
order by c.customer_id asc;
```

Result: The average customer spending is 922.87. The query identifies customers whose total spending exceeds this benchmark. Among the first 10 displayed customers, spending ranges from 1,089.56 to 2,448.98, with all shown customers exceeding the average.

	customer_id character varying (20)	customer_name character varying (100)	total_spending numeric	avg_spending numeric
1	CUST000003	Aditya Gupta	1116.91	922.87
2	CUST000004	Sahil Patel	1819.40	922.87
3	CUST000005	Pooja Kapoor	1089.56	922.87
4	CUST000006	Aarav Joshi	1181.86	922.87
5	CUST000008	Ritika Joshi	2032.83	922.87
6	CUST000008	Karan Sharma	1896.83	922.87
7	CUST000009	Pooja Mehta	2094.69	922.87
8	CUST000010	Anjali Reddy	1166.38	922.87
9	CUST000011	Kabir Gupta	2252.96	922.87
10	CUST000012	Arjun Singh	2448.98	922.87

Business Insight: The analysis establishes 922.87 as the benchmark for average customer spending and identifies customers who spend above this level. The displayed results show considerable variation in spending among above-average customers, with Arjun Singh recording the highest spending at 2,448.98—approximately 2.65 times the average customer spending.

Q15. Find the top 5 customers based on total spending and calculate their percentage contribution to overall revenue.

Analytical Approach: Ranks customers by total spending and calculates each selected customer's share of overall revenue.

```
select customer_id, customer_name, sum(total_amount) as total_spending,
round(sum(total_amount) * 100.0 / sum(sum(total_amount)) over (), 2) as percentage_contribution
from amazon_sales
group by customer_id, customer_name
order by total_spending desc
limit 5;
```

Result: Pooja Patel was the highest-spending customer, with total spending of 5,436.04, followed by Neha Sharma at 5,124.43 and Vikas Mehta at 5,003.65. Each of the top five customers contributed approximately 0.01% of the company's overall revenue.

	customer_id character varying (20)	customer_name character varying (100)	total_spending numeric	percentage_contribution numeric
1	CUST010696	Pooja Patel	5436.04	0.01
2	CUST035973	Neha Sharma	5124.43	0.01
3	CUST009614	Vikas Mehta	5003.65	0.01
4	CUST018720	Arjun Kapoor	4773.18	0.01
5	CUST022824	Pooja Joshi	4755.48	0.01

Business Insight: The top five customers have relatively similar spending levels, with Pooja Patel leading at 5,436.04. However, each customer's individual contribution to overall revenue is only approximately 0.01%, indicating that revenue is broadly distributed across the customer base rather than concentrated among a small number of customers.

Q18. Identify each customer's biggest purchase.

Analytical Approach: Aggregates each order per customer and ranks orders within each customer to find the largest.

```sql
with customer_table as(
    select
        customer_id, customer_name, order_id,
        sum(total_amount) as highest_order,
        row_number() over (
            partition by customer_id, customer_name
            order by sum(total_amount) desc
        ) as sales_rank
    from amazon_sales
    group by customer_id, customer_name, order_id
)
select *
from customer_table
where sales_rank = 1;
```

Result: The analysis identified the highest-value order placed by each customer by ranking their orders based on total_amount. The output returns the largest individual purchase for each customer grouping.

	customer_id character varying (20)	customer_name character varying (100)	order_id character varying (20)	highest_order numeric	sales_rank bigint
1	CUST000001	Sunita Kumar	ORD0085393	446.48	1
2	CUST000002	Arjun Singh	ORD0010091	689.40	1
3	CUST000002	Ritika Patel	ORD0040217	423.27	1
4	CUST000002	Vikas Gupta	ORD0006330	277.26	1
5	CUST000003	Aditya Gupta	ORD0005589	1116.91	1
6	CUST000004	Karan Kapoor	ORD0022455	583.70	1
7	CUST000004	Priya Sharma	ORD0085242	256.96	1
8	CUST000004	Sahil Patel	ORD0036461	1819.40	1
9	CUST000005	Pooja Kapoor	ORD0062298	1089.56	1
10	CUST000006	Aarav Joshi	ORD0098217	1181.86	1

Business Insight: The analysis helps identify customers' maximum transaction values, providing a view of high-value purchasing behavior at the individual customer level. This can help distinguish customers who make occasional large purchases from those whose spending is distributed across multiple smaller orders.

Q19. Find customers who purchased in at least two consecutive months.

Analytical Approach: The analysis first converts each customer's order activity into distinct customer-month combinations to avoid counting multiple orders within the same month separately. The LAG() window function is then used to identify each customer's previous active month. Customers are considered to have consecutive-month activity when their current purchase month is exactly one month after their previous active month.

```sql
with customer_months as(
    select distinct customer_id, customer_name,
    date_trunc('month', order_date)::date as month
    from amazon_sales
    order by month
    ),
monthly_history as (
    select customer_id, customer_name, month,
    lag(month) over (partition by customer_id,
    customer_name order by month) as previous_month
    from customer_months
)
select customer_id, customer_name, month,
previous_month from monthly_history
where previous_month is not null
and month = (previous_month + interval '1 month')::date;
```

Result: The analysis identified 14 customer-period records showing purchases in two consecutive calendar months. For example, Sneha Mehta purchased in February 2024 and again in March 2024, while Karan Kapoor purchased in June 2023 and again in July 2023. Aditya Verma appears twice, representing two separate consecutive-month periods: July–August 2023 and November–December 2024.

	customer_id character varying (20)	customer_name character varying (100)	month date	previous_month date
1	CUST005021	Sneha Mehta	2024-03-...	2024-02-01
2	CUST013654	Karan Kapoor	2023-07-...	2023-06-01
3	CUST015812	Priya Singh	2024-08-...	2024-07-01
4	CUST017666	Aarav Gupta	2021-02-...	2021-01-01
5	CUST028186	Aman Patel	2023-05-...	2023-04-01
6	CUST028957	Sunita Reddy	2023-01-...	2022-12-01
7	CUST036574	Aarav Joshi	2021-07-...	2021-06-01
8	CUST038927	Ritika Gupta	2021-04-...	2021-03-01
9	CUST039555	Aditya Gupta	2024-08-...	2024-07-01
10	CUST045442	Aditya Singh	2021-07-...	2021-06-01
11	CUST046106	Sunita Singh	2022-05-...	2022-04-01
12	CUST046930	Anjali Singh	2023-08-...	2023-07-01
13	CUST046941	Aditya Verma	2023-08-...	2023-07-01
14	CUST048999	Aditya Verma	2024-12-...	2024-11-01

Business Insight: Consecutive-month purchasing indicates recurring customer engagement across adjacent months. These customer-period records can be considered potential targets for retention initiatives, personalized offers, and loyalty campaigns, as they demonstrate continued purchasing activity without a gap between the identified months.

Q22. Which customers placed their second recorded order within 30 days of their first recorded order?

Analytical Approach: The analysis first orders each customer's recorded purchases chronologically and assigns an order sequence using ROW_NUMBER(). The LAG() function then retrieves the previous order date for each customer. By filtering for order_rank = 2, the analysis isolates each customer's second recorded order in the dataset and compares its date with the first recorded order date. Customers whose second recorded order occurred within 30 days are then returned, along with the number of days between the two purchases.

```
with ranked_orders as (
    select
        customer_id, customer_name, order_id, order_date,
        row_number() over (
            partition by customer_id, customer_name
            order by order_date, order_id
        ) as order_rank,
        lag(order_date) over (
            partition by customer_id, customer_name
            order by order_date, order_id
        ) as previous_order_date
    from amazon_sales
)
select
    customer_id, customer_name,
    previous_order_date as first_order_date,
    order_date as second_order_date,
    order_date - previous_order_date as days_between_orders
from ranked_orders
where order_rank = 2
  and order_date <= previous_order_date + interval '30 days'
order by days_between_orders;
```

Result: The analysis identified 14 customers whose second recorded order in the dataset was placed within 30 days of their first recorded order. The time between purchases ranged from 0 to 29 days. The shortest interval was recorded by Arjun Kumar, whose first and second recorded orders occurred on the same date, resulting in 0 days between purchases. The longest qualifying interval was 29 days, recorded by Sunita Reddy. Several customers placed their second recorded order within the first two weeks, including Aman Sharma (1 day), Aarav Gupta (9 days), and Anjali Patel (10 days).

	customer_id character varying (20)	customer_name character varying (100)	first_order_date date	second_order_date date	days_between_orders integer
1	CUST013277	Arjun Kumar	2022-12-25	2022-12-25	0
2	CUST009825	Aman Sharma	2023-01-29	2023-01-30	1
3	CUST017666	Aarav Gupta	2021-01-28	2021-02-06	9
4	CUST015145	Anjali Patel	2023-10-13	2023-10-23	10
5	CUST024836	Rohit Sharma	2021-09-11	2021-09-24	13
6	CUST014335	Arjun Reddy	2021-09-04	2021-09-19	15
7	CUST005431	Simran Kapoor	2023-12-16	2023-12-31	15

8	CUST007777	Neha Sharma	2023-10-07	2023-10-24	17
9	CUST045442	Aditya Singh	2021-06-19	2021-07-06	17
10	CUST005021	Sneha Mehta	2024-02-26	2024-03-15	18
11	CUST046930	Anjali Singh	2023-07-24	2023-08-16	23
12	CUST028186	Aman Patel	2023-04-29	2023-05-24	25
13	CUST046941	Aditya Verma	2023-07-28	2023-08-22	25
14	CUST028957	Sunita Reddy	2022-12-14	2023-01-12	29

Business Insight: A group of customers made a second recorded purchase relatively soon after their initial purchase, with 14 customers returning within 30 days. This early repeat-purchase behavior can serve as a useful indicator of short-term customer engagement and can help identify customers for retention or follow-up campaigns.

Q23. Find customers whose total spending is greater than the average total spending of customers in their city.

Analytical Approach: The analysis first calculates total spending for each customer within their respective city. It then calculates the average customer spending separately for each city using these customer-level totals. Finally, the customer-level spending is compared with the corresponding city average, returning only customers whose spending exceeds the average for their city.

```sql
with customer_spending as (
    select
        customer_id, customer_name, city,
        sum(total_amount) as total_spending
    from amazon_sales
    group by customer_id, customer_name, city
),
city_averages as (
    select
        city,
        round(avg(total_spending), 2) as city_average_spending
    from customer_spending
    group by city
)
select
    cs.customer_id,
    cs.customer_name,
    ca.city,
    cs.total_spending,
    ca.city_average_spending
from customer_spending cs
left join city_averages ca
    on cs.city = ca.city
where cs.total_spending > ca.city_average_spending
order by cs.customer_id;
```

Result: The query identifies customers whose total spending is above the average spending of customers in their respective city. Each returned record includes the customer's spending alongside the corresponding city-level average, allowing their spending to be evaluated relative to their local customer base.

	customer_id character varying (20)	customer_name character varying (100)	city character varying (50)	total_spending numeric	city_average_spending numeric
1	CUST000003	Aditya Gupta	San Jose	1116.91	914.53
2	CUST000004	Sahil Patel	Houston	1819.40	915.44
3	CUST000005	Pooja Kapoor	Chicago	1089.56	922.88
4	CUST000006	Aarav Joshi	Seattle	1181.86	925.71
5	CUST000008	Ritika Joshi	New York	2032.83	920.26
6	CUST000008	Karan Sharma	San Francisco	1896.83	915.23
7	CUST000009	Pooja Mehta	Jacksonville	2094.69	900.52
8	CUST000010	Anjali Reddy	Jacksonville	1166.38	900.52
9	CUST000011	Kabir Gupta	Indianapolis	2252.96	927.56
10	CUST000012	Arjun Singh	San Antonio	2448.98	917.60

Business Insight: Customers spending above their city average represent higher-value customers within their local markets and can be considered for targeted regional marketing and retention initiatives.

6. Geographic Analysis

Q14. Find the highest-selling product in every city.

Analytical Approach: Ranks products by quantity within each city and keeps the city leader.

```
with sales as (
    select city, product_id, product_name, sum(quantity) as quantity_sold,
    row_number() over (partition by city order by sum(quantity) desc) as rn
    from amazon_sales
    group by city, product_id, product_name
    )
select * from sales
where rn = 1;
```

Result: The analysis identified the highest-selling product in each city based on quantity sold. Phoenix recorded the highest city-level product sales with 414 units of the Mechanical Keyboard, while Washington recorded the lowest with 346 units of the Wireless Charger.

	city character varying (50)	product_id character varying (20)	product_name character varying (150)	quantity_sold bigint	rn bigint
1	Austin	P00038	External HDD 2TB	375	1
2	Charlotte	P00040	Microphone	382	1
3	Chicago	P00009	Portable SSD 1TB	363	1
4	Columbus	P00031	Kids Toy Car	364	1
5	Dallas	P00014	Drone Mini	393	1
6	Denver	P00036	Smart Light Bulb	377	1
7	Fort Worth	P00041	Webcam Full HD	380	1
8	Houston	P00015	Instant Pot	396	1
9	Indianapolis	P00005	Laptop Sleeve	350	1
10	Jacksonville	P00023	Cookware Set	406	1
11	Los Angeles	P00005	Laptop Sleeve	372	1
12	New York	P00011	Fitness Band	359	1
13	Philadelphia	P00006	Gaming Mouse	372	1
14	Phoenix	P00007	Mechanical Keyboard	414	1
15	San Antonio	P00023	Cookware Set	355	1
16	San Diego	P00047	Memory Card 128GB	361	1
17	San Francisco	P00025	Running Shoes	362	1
18	San Jose	P00001	Wireless Earbuds	387	1
19	Seattle	P00035	Desk Plant	385	1
20	Washington	P00048	Wireless Charger	346	1

Business Insight: The highest-selling products vary considerably by city, indicating that no single product dominates sales across all locations. Phoenix recorded the highest city-level product sales at 414 units for the Mechanical Keyboard, while Washington recorded the lowest at 346 units for the Wireless Charger. This variation suggests that product demand differs across cities.

7. Seller Analysis

Q17. Which sellers generate the highest revenue, and how many units do they sell?

Analytical Approach: Seller-level performance was analyzed by grouping the orders by seller_id. Total revenue was calculated by summing total_amount, while total units sold were calculated by summing quantity. The sellers were then ranked in descending order of revenue to identify the highest-revenue sellers.

```
select seller_id, sum(total_amount) as revenue,
sum(quantity) as quantity_sold
from amazon_sales
group by seller_id
order by revenue desc;
```

Result: SELL000806 generated the highest revenue among the displayed sellers, with 71,283.67 in revenue and 234 units sold. It was followed closely by SELL000440, which generated 71,062.59 from 197 units. The top 10 sellers in the query output generated between 68,591.86 and 71,283.67 in revenue, with unit sales varying across sellers.

	seller_id character varying (20)	revenue numeric	quantity_sold bigint
1	SELL00806	71283.67	234
2	SELL00440	71062.59	197
3	SELL01225	70785.92	204
4	SELL00709	70491.92	223
5	SELL00221	70446.50	211

Business Insight: Revenue among the top-performing sellers was relatively close, with only 2,691.81 separating the highest and lowest sellers in the top 10. Unit volumes varied across sellers, showing that similar revenue levels can be achieved with different sales volumes.

8. Time-Based Sales Analysis

Q12. Calculate the running (cumulative) revenue by order date.

Analytical Approach: First, the analysis creates daily revenue and then applies a cumulative window sum ordered by date.

```
with current_revenue as(
    select order_date, sum(total_amount)
        as daily_revenue from amazon_sales
    group by order_date
    )
select *, (sum(daily_revenue) over(order by order_date))
    as running_revenue from current_revenue;
```

Result: The analysis calculates daily revenue and progressively accumulates it over time. In the first 10 days of January 2020, cumulative revenue increased from 55,180.39 on January 1 to 511,429.02 on January 10.

	order_date date	daily_revenue numeric	running_revenue numeric
1	2020-01-01	55180.39	55180.39
2	2020-01-02	45415.31	100595.70
3	2020-01-03	40744.16	141339.86
4	2020-01-04	58572.99	199912.85
5	2020-01-05	43502.13	243414.98
6	2020-01-06	57544.74	300959.72
7	2020-01-07	48708.11	349667.83
8	2020-01-08	48340.22	398008.05
9	2020-01-09	51222.01	449230.06
10	2020-01-10	62198.96	511429.02

Business Insight: The running revenue increases continuously as daily revenue is accumulated over time. During the first 10 days of January 2020, cumulative revenue reached 511,429.02, with daily revenue fluctuating between 40,744.16 and 62,198.96. This demonstrates how cumulative revenue builds from individual daily sales and provides a clear view of revenue progression over the period.

Q20. Classify each order as either Weekday or Weekend. Then calculate Total Revenue, Total Orders, and Average Order Value (AOV) separately for each group.

Analytical Approach: The analysis uses EXTRACT(ISODOW FROM order_date) to identify the day of the week for each order. Days 6 (Saturday) and 7 (Sunday) are classified as Weekend, while the remaining days are classified as Weekday. The orders are then grouped by this classification to calculate total revenue, total order count, and average order value (AOV) for each day type.

```
select sum(total_amount) as total_revenue, count(order_id) as order_count,
    round(avg(total_amount), 2) as aov,
    case
        when extract(isodow from order_date) in (6, 7) then 'Weekend'
        else 'Weekday'
    end as day_type
    from amazon_sales
group by day_type;
```

Result: Weekdays generated substantially more revenue and orders than weekends, with 71,560 weekday orders generating 65.82M, compared with 28,440 weekend orders generating 26.01M.

The AOV was relatively similar between the two groups: 919.74 on weekdays vs 914.53 on weekends, a difference of only 5.21.

	total_revenue numeric	order_count bigint	aov numeric	day_type text
1	26009304.93	28440	914.53	Weekend
2	65816342.99	71560	919.74	Weekday

Business Insight: The dataset shows that weekday purchasing activity is considerably higher than weekend activity, resulting in substantially greater weekday revenue. However, the relatively small difference in AOV indicates that the revenue gap is driven primarily by order volume rather than customers spending substantially more per order on weekdays.

This could help inform inventory planning, staffing, and promotional scheduling, with greater operational demand occurring during weekdays. Weekend promotions could potentially be evaluated separately if the business wants to increase weekend order volume.

Q21. Calculate monthly revenue and show:

Current month revenue, Previous month revenue, MoM growth %, Handle the first month appropriately.

Analytical Approach: The analysis first aggregates total sales revenue at the monthly level using DATE_TRUNC('month', order_date). A LAG() window function then brings the previous month's revenue alongside each month's current revenue. Finally, the month-over-month (MoM) growth percentage is calculated by comparing the current month's revenue with the previous month's revenue.

```
with monthly_rev as(
    select date_trunc('month', order_date)::date as month,
    sum(total_amount) as total_revenue
    from amazon_sales
    group by month
    ),
previous_rev as(
    select *, lag(total_revenue) over(order by month) as previous_revenue
    from monthly_rev
)
select month, total_revenue, previous_revenue,
round((100.0 * (total_revenue - previous_revenue) / previous_revenue), 2)
    as mom_growth_pct
from previous_rev
order by month;
```

Result: The analysis covers January 2020 through December 2024, providing a month-by-month view of revenue and growth.

- January 2020 generated 1,637,069.40. Since it is the first month in the dataset, there is no previous month available for comparison.
- February 2020 recorded the largest decline in the displayed period at -18.18%, with revenue falling to 1,339,388.21.
- March 2020 recovered strongly, increasing 18.50% to 1,587,125.66.
- The strongest positive MoM growth in the dataset was 18.50% in March 2020.
- Other notable increases include March 2022 (+16.06%), March 2023 (+12.43%), March 2024 (+9.64%), and May 2024 (+8.70%).
- The final month, December 2024, generated 1,451,073.77, representing a 5.15% decline from November 2024.

Overall, monthly revenue generally remained within a relatively consistent range of roughly 1.34M–1.64M, despite recurring month-to-month fluctuations.

	month date	total_revenue numeric	previous_revenue numeric	mom_growth_pct numeric
1	2020-01-...	1637069.40	[null]	[null]
2	2020-02-...	1339388.21	1637069.40	-18.18
3	2020-03-...	1587125.66	1339388.21	18.50
4	2020-04-...	1588347.29	1587125.66	0.08
5	2020-05-...	1533834.17	1588347.29	-3.43

Business Insight: Monthly revenue fluctuates throughout the period, with several sharp increases and declines. Notably, March recorded strong MoM growth in multiple years. This monthly trend can help the business identify periods of stronger or weaker sales and support future sales planning.

9. Revenue Concentration & Retention

Q24. Identify the top 10% of customers by total spending and determine what percentage of the company's total revenue they generate.

Analytical Approach: The analysis first calculates the total spending of each customer and ranks customers in descending order of spending using ROW_NUMBER(). The top 10% of customers are then identified dynamically using CEIL(COUNT(*) * 0.10). Finally, the combined spending of these top customers is compared with the company's total revenue using a window function and a subquery to determine their overall revenue contribution.

```sql
with customer_totals as (
    select
        customer_id,
        customer_name,
        sum(total_amount) as total_spending,
        row_number() over (
            order by sum(total_amount) desc
        ) as customer_rank
    from amazon_sales
    group by customer_id, customer_name
),
top_customers as (
    select *
    from customer_totals
    where customer_rank <= (
        select ceil(count(*) * 0.10)
        from customer_totals
    )
)
select
    customer_id,
    customer_name,
    total_spending,
    customer_rank,
    round(
        sum(total_spending) over () * 100.0 /
        (select sum(total_amount) from amazon_sales),
        2
    ) as revenue_contribution_pct
from top_customers
order by customer_rank;
```

Result: The analysis identified the top 10% of customers by total spending, whose combined spending represents 26.74% of the company's total revenue. The highest-ranked customer spent 5,436.04, followed by customers spending 5,124.43 and 5,003.65 respectively.

	customer_id character varying (20)	customer_name character varying (100)	total_spending numeric	customer_rank bigint	revenue_contribution_pct numeric
1	CUST010696	Pooja Patel	5436.04	1	26.74
2	CUST035973	Neha Sharma	5124.43	2	26.74
3	CUST009614	Vikas Mehta	5003.65	3	26.74
4	CUST018720	Arjun Kapoor	4773.18	4	26.74
5	CUST022824	Pooja Joshi	4755.48	5	26.74
6	CUST008882	Sneha Reddy	4730.47	6	26.74
7	CUST022031	Priya Kapoor	4266.02	7	26.74
8	CUST031449	Vivaan Kapoor	4253.27	8	26.74
9	CUST005523	Sahil Patel	4239.18	9	26.74
10	CUST047013	Arjun Kumar	4222.62	10	26.74

Business Insight: The top 10% of customers generate 26.74% of total revenue, showing that a relatively small segment of high-value customers contributes a substantial share of sales.

Q25. Which calendar month generated the highest returning-customer revenue, based on the customer's first valid purchase month recorded in the dataset?

Analytical Approach: The analysis first identifies each customer's first valid purchase using only Delivered and Shipped orders. Each valid order is then classified as New if it occurs in the customer's first purchase month, or Returning if it occurs in a later month.

Monthly revenue is then separated into new-customer and returning-customer revenue. Finally, DENSE_RANK() ranks the months by returning-customer revenue, allowing the query to return all months tied for the highest value.

```
with first_purchases as (
    select
        customer_id,
        customer_name,
        min(order_date) as first_purchase_date
    from amazon_sales
    where order_status in ('Delivered', 'Shipped')
    group by customer_id, customer_name
),
valid_orders as (
    select
        customer_id,
        customer_name,
        order_date,
        total_amount
    from amazon_sales
    where order_status in ('Delivered', 'Shipped')
),
customer_types as (
    select
        fp.customer_id,
        fp.customer_name,
        vo.order_date,
        fp.first_purchase_date,
        vo.total_amount,
        case
            when date_trunc('month', vo.order_date)::date >
                 date_trunc('month', fp.first_purchase_date)::date
            then 'Returning'
            else 'New'
        end as customer_type
    from first_purchases fp
    join valid_orders vo
        on fp.customer_id = vo.customer_id
       and fp.customer_name = vo.customer_name
),
```

```sql
monthly_revenue as (
    select
        date_trunc('month', order_date)::date as month,
        sum(case
            when customer_type = 'New' then total_amount
            else 0
        end) as new_revenue,
        sum(case
            when customer_type = 'Returning' then total_amount
            else 0
        end) as returning_revenue,
        sum(total_amount) as total_revenue
    from customer_types
    group by month
),
ranked_months as (
    select
        *,
        dense_rank() over (
            order by returning_revenue desc
        ) as revenue_rank
    from monthly_revenue
)
select
    month,
    new_revenue,
    returning_revenue,
    total_revenue
from ranked_months
where revenue_rank = 1
order by month;
```

Result: September 2024 generated the highest returning-customer revenue:

Returning customers generated 20,254.70, accounting for approximately 1.55% of September's total revenue.

	month date	new_revenue numeric	returning_revenue numeric	total_revenue numeric
1	2024-09-01	1288732.81	20254.70	1308987.51

Business Insight: September 2024 recorded the highest returning-customer revenue at 20,254.70, while most of the month's revenue came from customers in their first recorded purchase month. This indicates that, within this dataset's recorded purchase history, returning-customer revenue represented a relatively small share of monthly sales.

SQL Skill Story

Foundation: SELECT, COUNT(), SUM(), AVG(), filtering with WHERE

Aggregation: GROUP BY, HAVING, DISTINCT

Conditional Logic: CASE, EXTRACT()

Modular Analysis: Common Table Expressions (CTEs)

Window Analysis: LAG(), ROW_NUMBER(), OVER(), PARTITION BY, windowed SUM()

Time-Series Analysis: DATE_TRUNC(), INTERVAL, date arithmetic, month-over-month comparisons

Comparative Analysis: LEFT JOIN, CROSS JOIN

Business Analytics: Repeat Customers, Revenue Concentration, City Benchmarking, New vs. Returning Customer Revenue, Customer Retention

Assumptions & Analytical Boundaries

- Revenue is represented by total_amount, as used throughout the supplied queries.
- **Q20** treats Saturday and Sunday as weekend days using EXTRACT(ISODOW FROM order_date).
- **Q24** identifies the highest-spending 10% of customers through customer-level ranking.
- **Q25** treats Delivered and Shipped orders as valid purchases.
- **Q25** defines a customer's first purchase as their earliest valid purchase recorded within the dataset.
- The dataset covers **January 2020 through December 2024**; therefore, "first-ever purchase" refers to the customer's first recorded valid purchase **within this dataset**, not necessarily their first purchase from Amazon historically.
- Where a query returns multiple records, displayed rows may represent a selected portion of the complete output for reporting readability.
- Numerical findings are reported only where verified query outputs are available; unsupported KPIs or conclusions are not inferred.

Business Insight Framework

The project uses five core management questions to translate SQL analysis into business insights:

1. **What drives revenue?**
   Examine overall sales, order volume, products, categories and payment methods.
2. **Which products and customers matter most?**
   Identify high-volume products, high-value customers and customer reach.
3. **Where and when is performance changing?**
   Analyse cities, sellers and monthly sales patterns.
4. **How concentrated is revenue?**
   Examine high-value customers and their contribution to overall revenue.
5. **How much revenue is sustained by returning customers?**
   Separate new-customer and returning-customer revenue to understand customer retention patterns.

Conclusion

The SQL project demonstrates a progression from foundational querying and aggregation to advanced analytical techniques, including CTEs, window functions, time-series analysis, customer segmentation and business-oriented comparisons. The analysis applies these techniques to practical business questions involving sales performance, product demand, customer behavior, geographic performance, revenue concentration and customer retention.